
The Machine Learning Workflow & Data

Contact Session 2 — Module 2

Before any clever method can work, the data has to be clean and fair. This is the quiet, unglamorous 80% of every project — explained simply.

- | | |
|------------------|------------------|
| ■ Intuition | ■ Analogy |
| ■ Worked example | ■ Solved problem |
| ■ Common pitfall | ■ Key takeaways |

Contents

1	Why this whole session matters	3
2	The big picture: the workflow	4
3	Getting to know your data	5
3.1	Not all columns are the same <i>kind</i> of number	5
4	Dirty data, and how to clean it	6
4.1	Filling in missing values	6
4.2	Spotting outliers — two simple tests	6
5	Taking a fair sample	8
6	Putting features on the same scale	9
7	Making better features (often the real win)	10
8	The whole story on one page	11

1 Why this whole session matters

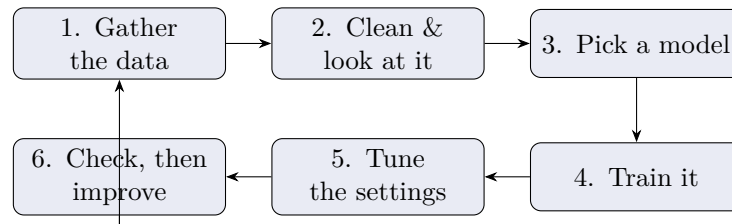
There's an old joke: data scientists spend 80% of their time cleaning data, and the other 20% complaining about cleaning data. It's only half a joke. The fanciest method in the world cannot fix bad ingredients.

A model never sees the real world. It only sees **the table of numbers you hand it**. If that table is wrong — bad values, missing rows, an unfair sample — the model faithfully learns the wrong thing. Garbage in, garbage out.

So this session is everything that happens *before* training: understanding your data, fixing its flaws, taking a fair sample, and shaping good features.

2 The big picture: the workflow

Before touching data, ask yourself four quick questions: *Is there really a pattern here? Could I just solve this with a plain formula instead? Do I actually have data? Is machine learning the right tool at all?* If the answers say “go,” the work flows in a loop:



It’s a loop, not a line — you go around again and again, getting a little better each time. Most of *this* session lives in steps 1 and 2.

3 Getting to know your data

A dataset is just a table. Each **row** is one thing (a customer, a house). Each **column** is one fact about it (age, price). The textbook calls rows *objects* or *instances* and columns *attributes* or *features* — same thing.

3.1 Not all columns are the same *kind* of number

This sounds picky, but it decides what maths you’re allowed to do. There are four kinds of column:

Kind	What it is	Examples
Names	labels with no order	colour, city, ID number
Ranks	ordered, but gaps don’t mean much	{small, medium, large}, grades
Even-gap, no real zero	equal steps, but “zero” is arbitrary	temperature in °C, calendar dates
Even-gap, real zero	equal steps <i>and</i> a true zero	height, weight, counts, price

! Common Pitfall

20°C is *not* “twice as hot” as 10°C, because in Celsius zero is just a chosen spot, not truly nothing. But 20 kg *is* twice 10 kg, because zero kg really means no weight. Knowing the kind of column tells you which comparisons make sense — and which would be nonsense.

Columns are also either **whole-number-ish** (counts, zip codes — only certain values) or **smoothly varying** (height, weight — any value in a range).

4 Dirty data, and how to clean it

Real data is messy. The usual troublemakers:

- **Noise** — random junk that fuzzes a value (like static on a phone call).
- **Outliers** — a few values wildly different from the rest.
- **Missing values, duplicates, and contradictions** (Age says 42, but Birthday says 2010).

★ One person's junk is another's treasure

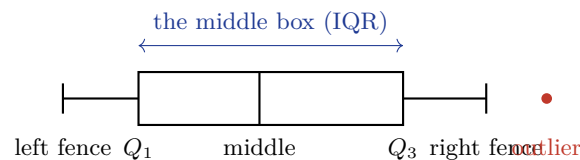
Is a Rs. 1,000,000 transaction an outlier to throw away, or the most important row in the table? It depends entirely on *why you're looking*. To a sales model it's a typo to remove; to a fraud detector it's the whole point. So always decide *what you're trying to do* before you decide what to clean.

4.1 Filling in missing values

Rather than delete rows with a blank, we usually *fill the blank in* with a sensible stand-in: the column's average (for numbers), or its most common value (for categories). If the column has wild outliers, use the middle value (the median) instead of the average — it's harder to fool.

4.2 Spotting outliers — two simple tests

Test 1: the “middle box” rule. Line the values up in order. The middle half of them sits in a box. Call the box's left edge Q_1 and its right edge Q_3 , and its width $IQR = Q_3 - Q_1$. Anything sitting more than 1.5 box-widths beyond either edge is flagged as an outlier.



Σ Find the outliers

Values: 10, 2, 11, 15, 11, 14, 13, 17, 12, 22, 14, 11. Put them in order:

2, 10, 11, 11, 11, 12, 13, 14, 14, 15, 17, 22.

The box edges work out to $Q_1 = 11$ and $Q_3 = 14.5$, so the box width is $14.5 - 11 = 3.5$. The fences are:

$$\text{left} = 11 - 1.5(3.5) = 5.75, \quad \text{right} = 14.5 + 1.5(3.5) = 19.75.$$

Anything below 5.75 or above 19.75 is an outlier. So **2** (too small) and **22** (too big) get flagged — with 22 being the obvious odd one out.

Test 2: the “three steps from the middle” rule. If the values form a nice bell shape, almost all of them (99.7%) sit within three “typical steps” (standard deviations) of the average. Anything further out is suspicious.

Σ Three steps

If the average is 50 and one typical step is 5, then almost everything lives between $50 - 3(5) = 35$ and $50 + 3(5) = 65$. A reading of 70 is more than three steps out, so it’s an outlier; a reading of 48 is fine.

! Common Pitfall

The “three steps” rule only works if the data is bell-shaped. On lopsided data (like income, where a few people earn a fortune) it wrongly flags genuine high earners. For lopsided data, trust the “middle box” rule instead.

5 Taking a fair sample

We rarely use *all* the data — we take a sample. The golden rule:

★ A fair sample beats a big one

A small sample is fine *as long as it looks like the whole*. The danger isn't smallness — it's *unfairness*. If your sampling method quietly favours one type of case, then even a huge sample is biased, and no amount of extra data fixes it. A shaky hand (small sample) just needs more data; a crooked ruler (biased method) needs to be straightened.

The safe everyday choice is a **balanced sample**: if your data has three flower types in equal numbers, make sure your sample keeps them in equal numbers too.

Σ Why balance matters

A flower dataset has 50 each of three types (150 total). You want a 100/50 train/test split. A *careless* random split might hand you 38/28/34 for training — the types drift out of balance, quietly biasing everything. A *balanced* split keeps each type even (about 33/33/34), so training and testing both stay fair.

When one class is rare (say, fraud is 1 in 1000), we *rebalance* on purpose: keep fewer of the common class, or make extra copies (or synthetic look-alikes) of the rare one. And we always split data into three piles: one to **train** on, one to **tune** on, and one kept locked away to **honestly test** at the very end.

6 Putting features on the same scale

Here’s a trap. Suppose two facts are *age* (around 30) and *income* (around 500,000). Many methods measure “how different are two people” by these numbers, and income’s huge size drowns age out completely — age might as well not exist.

The fix is **scaling**: shrink every column to a similar size so each gets a fair say. Two common ways, each just one little formula:

- **Squeeze into 0–1** (normalisation): $v' = \frac{v - \min}{\max - \min}$. The smallest value gives 0 (top and bottom cancel), the biggest gives 1, everything else lands in between — the formula just asks “how far along the range is this value?”
- **Centre and standardise**: $v' = \frac{v - \text{average}}{\text{typical step}}$. Subtracting the average slides the middle to 0; dividing by the typical step (the standard deviation) rescales so “ $v' = 2$ ” always means “two typical steps above average,” whatever the column.

Σ Both, on one value

Squeeze-to-0–1: if income runs from Rs. 12,000 to Rs. 98,000, then Rs. 73,000 lands at $\frac{73000-12000}{98000-12000} = \frac{61000}{86000} \approx 0.71$ of the way up.

Standardise: if the average is Rs. 54,000 and a typical step is Rs. 16,000, then Rs. 73,600 sits at $\frac{73600-54000}{16000} = 1.225$ steps above average.

! Common Pitfall

Work out the scaling using *only the training data*, then apply that same recipe to the test data. If you peek at the test data while scaling, you’ve secretly leaked the answer key into training, and your reported score will look better than it really is.

7 Making better features (often the real win)

Good features beat fancy methods. There are four everyday moves:

1. **Trim down (selection).** Drop columns that are useless, repeated, or mostly blank. Fewer, better facts means faster, cleaner learning.
2. **Combine (construction).** Build a smarter fact out of raw ones — e.g. `days as customer = last visit - first visit`. One good combined fact can be worth ten raw ones.
3. **Shrink the width (extraction).** When there are *too many* columns, squash them into a few summary columns (a method called PCA does this). Why bother? Because of the curse below.

~> The curse of too many facts

Finding a friend on a 100 m line is easy. On a 100 m field, harder. In a giant 3-D hall, harder still. In a space with 100 different facts, everyone is so spread out that the “nearest” person is almost as far as the farthest — the idea of “close” stops meaning anything. Too many columns dilute your data until patterns vanish. That’s why we sometimes shrink the width.

4. **Reshape (transformation).** Turn a smooth number into buckets (age → child / adult / senior), or turn words into numbers so a model can read them. For words: give each category its *own yes/no column* (“one-hot”), unless the categories have a real order, in which case a simple 0,1,2 ranking is fine.

! Common Pitfall

If you number unordered categories as Red= 0, Green= 1, Blue= 2, the model wrongly thinks Blue is “more” than Red. For categories with no real order, give each its own yes/no column instead.

8 The whole story on one page

★ Key Takeaways

- The model only sees your table — so make the table honest.
- Know your **kinds of columns** (names, ranks, even-gap with or without a real zero); the kind decides what maths is allowed.
- Clean the mess: fill blanks sensibly, and flag outliers with the **middle-box** rule (works always) or the **three-steps** rule (only for bell-shaped data).
- Take a **fair, balanced sample**; a biased method can't be saved by more data. Keep separate train / tune / test piles.
- **Scale** every feature to a similar size, and fit the scaling on *training data only*.
- **Feature work** — trim, combine, shrink, reshape — usually helps more than switching methods.

Where this comes from: Tan, Steinbach & Kumar, *Introduction to Data Mining*, Ch. 2–3. Course slides M2 (Dr. Neha Vinayak, BITS Pilani WILP).